A D PATEL INSTITUTE OF TECHNOLOGY
(The Charutar Vidya Mandal University)
GSIRE An ISO 9001:2015 Certified Institute

Smart Patient Queue System

A Project report submitted
In partial fulfilment of the requirements for the Degree of
Bachelor of Technology

In
Information Technology
Semester – V
Subject: Design and Analysis of Algorithms

By

Savani Ayushi Jivrajbhai (12402080601014)

Faculty Guide
Nayan Mali
Assistant Professor

Academic Year 2026-27 (Odd)

Department of Information Technology
A D Patel Institute of Technology
New Vallabh Vidyanagar
Vitthal Udyognagar - 388121
Anand (Gujarat), India

CERTIFICATE

This is to certify that the project work embodied in this report entitled, "**Smart Patient Queue System: Emergency Prioritization for Hospital Staff**" was carried out by **Savani Ayushi Jivrajbhai (12402080601014)** at A D Patel Institute of Technology for partial fulfilment of B.Tech. degree to be awarded by the Charutar Vidya Mandal (CVM) University, in the subject Design and Analysis of Algorithms. This project work has been carried out under my supervision and is to the satisfaction of the department.

Date:

Place:

Guide
Nayan Mali
Assistant Professor

Head of Department
Dr. Priyang Bhatt
Professor & Head

Acknowledgement

I would like to express my sincere thanks to my respected faculty guide for guiding me throughout the preparation of this Smart Patient Queue project and report. Their support, advice and encouragement helped me to understand the Design and Analysis of Algorithms subject better and to complete my work successfully.

I am also thankful to A D Patel Institute of Technology and the Department of Information Technology for providing me with the opportunity and the resources to complete this project. The learning environment and the lab facilities made it easier to build and test the application.

I would like to thank my parents for their constant support and motivation. Their encouragement helped me stay focused and complete this work on time.

I am also grateful to my friends and classmates who helped me by sharing ideas and supporting me whenever needed.

Lastly, I would like to thank all those who directly or indirectly helped me in completing this project. It has been a great learning experience for me.

Savani Ayushi Jivrajbhai
(12402080601014)

Abstract

Hospitals often experience difficulties in managing patient queues, especially when a large number of patients arrive within a short period. Manual queue management can result in long waiting lines, misplaced records, difficulty identifying emergency cases, and delays in retrieving patient information. A centralized system is required to register patients, organize their queues according to priority, and track their consultation status efficiently.

The **Smart Patient Queue System** is a web-based application developed to simplify patient registration and queue management for hospital staff. The system provides a centralized dashboard through which authorized staff members can log in, register patients, assign priority levels, search patient records, filter queues by department, and update consultation status.
The application uses three priority levels: Emergency, Urgent, and Normal. Patients marked as Emergency are placed ahead of Urgent and Normal patients. When multiple patients have the same priority, their registration time is used to maintain arrival order. Each patient receives a unique automatically generated queue number, allowing staff to identify and manage records conveniently.

The system is developed using HTML5, CSS3, and JavaScript for the frontend; Python and Flask for the backend; Jinja2 for dynamic HTML rendering; and MySQL for database management. Staff authentication uses sessions and password hashing. Parameterized SQL queries, CSRF tokens, browser-side and server-side validation, and secure cookie settings are included as security measures.

The system aims to reduce manual paperwork, improve the visibility of emergency cases, simplify patient record retrieval, and provide clear tracking of consultation progress. It is designed as a demonstration application for a single hospital environment. Future improvements may include doctor-specific access, SMS and WhatsApp notifications, waiting-time estimation, appointment management, analytics, and cloud deployment.

Keywords: Smart Patient Queue System, Hospital Queue Management, Priority Queue, Patient Registration, Emergency Prioritization, Flask, Python, MySQL, Web Application, Database Security.

Table of Content

Certificate

Acknowledgement

Abstract

Table of Contents

CHAPTER – 1 INTRODUCTION

1.1 Background
1.2 Problem Statement
1.3 Objectives of the Project
1.4 Scope of the Project

CHAPTER – 2 THEORETICAL BACKGROUND

2.1 Patient Queue Management and Basic Terms
2.2 Priority-Based Queue Management
2.3 Web Application Architecture
2.4 Relational Database Management System
2.5 Security Concepts Used in the System

CHAPTER – 3 SYSTEM DESIGN

3.1 System Architecture
3.2 Data Design
3.3 Graphical User Interface Design
3.4 System Workflow and Flowchart

CHAPTER – 4 IMPLEMENTATION

4.1 Technologies Used
4.2 Main Modules of the Application
4.3 Important Implementation Concepts
4.4 Validation, Security and Error Handling

CHAPTER – 5 WORKED EXAMPLE

CHAPTER – 6 RESULTS AND SCREENSHOTS

6.1 Application Screenshots
6.2 Testing
6.3 Comparison of Manual and Smart Queue Management

CHAPTER – 7 COMPLEXITY ANALYSIS

CHAPTER – 8 ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

8.1 Advantages
8.2 Limitations
8.3 Future Scope

CHAPTER – 9 CONCLUSION

REFERENCES

CHAPTER – 1 INTRODUCTION

1.1 Background

Hospitals receive many patients every day, making queue management an important part of their operations. Manual registration and paper-based queues can create delays, make emergency cases difficult to identify, and increase the time required to locate patient records.

The Smart Patient Queue System provides a centralized web application to manage patient registration, queue prioritization, record searching, and consultation tracking.

1.2 Problem Statement

Manual queue management may cause long waiting lines, difficulty prioritizing emergencies, misplaced records, and a lack of centralized monitoring.

Proposed Solution: The system registers patients, assigns priority levels, generates unique queue numbers, organizes the queue, and tracks consultation status through a single dashboard.

1.3 Objectives of the Project

- Reduce manual paperwork.
- Provide secure staff login.
- Register patients and generate unique queue numbers.
- Prioritize Emergency, Urgent, and Normal cases.
- Search patients by name or ID.
- Filter patients by department.
- Track consultation status until completion.

1.4 Scope of the Project

The project includes patient registration, staff authentication, priority-based queue organization, searching, department filtering, and consultation-status tracking. Multi-hospital management, notifications, appointment scheduling, waiting-time estimation, and advanced analytics are outside the current scope.

CHAPTER – 2 THEORETICAL BACKGROUND

2.1 Priority-Based Queue Management

A traditional FIFO queue serves people in arrival order. A priority queue organizes records according to assigned importance.
The Smart Patient Queue System uses three priorities:

1. Emergency
2. Urgent
3. Normal

Patients with equal priority are ordered by registration time. Priority is selected by hospital staff; the system does not diagnose patients.

2.2 Web Application Architecture

The application follows a client-server architecture. The browser displays the interface, Flask processes requests, Jinja2 renders dynamic pages, and MySQL stores patient and staff records.

2.3 Database Management

MySQL stores data in two main tables: patients and staff_accounts. Primary keys identify records, unique constraints prevent duplicate queue numbers and account identifiers, and restricted values maintain valid priority and status categories.

2.4 Security Concepts

The application uses session-based authentication, password hashing, parameterized SQL queries, CSRF tokens, browser-side and server-side validation, and HttpOnly and SameSite cookie settings.

CHAPTER – 3 SYSTEM DESIGN

3.1 System Architecture

The system consists of five main components:

1. **Hospital Staff:** Uses the application through a browser.
2. **Frontend:** HTML5, CSS3, and JavaScript provide forms, tables, and dashboard controls.
3. **Backend:** Python and Flask handle routes, sessions, validation, and requests.
4. **Templates:** Jinja2 renders dynamic HTML pages.
5. **Database:** MySQL stores patient and staff information.

Architecture Flow: Staff → Web Interface → Flask Backend → MySQL Database → Dynamic Response through Jinja2.

3.2 Database Design

Patients Table: patient_id, patient_name, age, gender/contact, department, symptoms, priority_level, queue_number, status, and registered_at.

Staff Accounts Table: staff_id, username, email, password_hash, and created_at.
The patient ID is the primary key. Queue numbers, usernames, and emails are unique. Priority values are Emergency, Urgent, and Normal; status values are Waiting, In Consultation, and Completed.

3.3 User Interface

The interface includes:

- Staff login page
- Central dashboard
- Patient registration form
- Active queue table
- Patient search and department filter
- Consultation status controls
- Light and dark display modes

3.4 Workflow

Start → Staff Login → Validate Credentials → Dashboard → Register Patient → Assign Priority → Generate Queue Number → Display Queue → Search or Filter → Update Status → Completed.

CHAPTER – 4 IMPLEMENTATION

4.1 Technologies Used

Technology	Purpose
HTML5, CSS3, JavaScript	Frontend and interface interactions
Python	Backend logic
Flask	Routing, sessions, and request handling
Jinja2	Dynamic HTML rendering
MySQL	Data storage
MySQL Connector	Database access
python-dotenv	Environment configuration

4.2 Main Modules

Authentication: Validates staff credentials and restricts access to protected pages.
Patient Registration: Collects patient details and stores the record in MySQL.
Priority Management: Organizes patients according to Emergency, Urgent, and Normal priority.
Queue Management: Generates unique queue numbers and displays the organized queue.
Search and Filtering: Locates records using patient name or ID and filters by department.
Status Management: Tracks patients as Waiting, In Consultation, or Completed.

4.3 Queue Ordering Logic

The queue can be organized using the following SQL ordering:

```
ORDER BY CASE priority_level
   WHEN 'Emergency' THEN 1
   WHEN 'Urgent' THEN 2
   ELSE 3
END,
registered_at ASC,
patient_id ASC;
```

This places Emergency patients first, followed by Urgent and Normal patients. Registration time maintains the order among patients with equal priority.

4.4 Security and Validation

The system uses password hashing, authenticated sessions, parameterized SQL queries, CSRF tokens, input validation, and secure cookie settings. Environment configuration is managed using python-dotenv.

CHAPTER – 5 WORKED EXAMPLE

Consider four registered patients:

Patient	Time	Priority
Aarav Mehta	09:08 AM	Urgent
Mohan Joshi	09:31 AM	Emergency
Vivaan Trivedi	09:58 AM	Normal
Imran Vora	10:15 AM	Urgent

The final queue order is:

1. Mohan Joshi – Emergency
2. Aarav Mehta – Urgent
3. Imran Vora – Urgent
4. Vivaan Trivedi – Normal

Although Mohan arrived after Aarav, he is placed first because Emergency has higher priority. Aarav is placed before Imran because both have Urgent priority and Aarav registered earlier. This example demonstrates how priority and registration time determine the queue order.

CHAPTER – 6 RESULTS AND SCREENSHOTS

The Smart Patient Queue System provides a centralized platform for managing patient registration and hospital queues. It combines patient information, priority assignment, queue number generation, department filtering, and consultation-status tracking in a single application. The main results and observations are as follows:

- **Patient Registration:** Staff can register patients by entering the required details.
- **Priority Management:** Patients are organized into Emergency, Urgent, and Normal categories.
- **Automatic Queue Number:** Each registered patient receives a unique queue number.
- **Queue Organization:** Emergency patients are placed before Urgent and Normal patients. Patients with equal priority are arranged according to registration time.
- **Patient Search:** Staff can search for records using a patient name or ID.
- **Department Filtering:** The queue can be filtered to display patients from a selected department.
- **Status Tracking:** Patient progress can be recorded using Waiting, In Consultation, and Completed statuses.
- **Centralized Dashboard:** Staff can access queue information and management functions from one interface.

These features demonstrate how the application can simplify queue organization and reduce dependence on manual record management.

6.2 Testing

Testing is required to verify that the application behaves according to its intended functionality. The following test cases cover the major modules of the Smart Patient Queue System.

Test Case	Expected Result
Staff login with valid credentials	Dashboard access is granted
Staff login with invalid credentials	Access is denied
Register a patient	Patient record is saved and a queue number is assigned
Assign Emergency priority	Patient is placed ahead of lower-priority patients
Register patients with equal priority	Earlier registration is placed first
Search by patient name or ID	Matching patient records are displayed
Apply department filter	Patients from the selected department are displayed
Update consultation status	The selected status is reflected in the record
Submit invalid form data	Validation prevents inappropriate submission
Access a protected page without login	Unauthenticated access is restricted

The table lists the expected outcomes of the test cases. Actual test results should be confirmed by executing these cases in the application.

6.3 Comparison with Manual Queue Management

Parameter	Manual System	Smart Patient Queue System
Registration	Paper-based	Web-based
Priority assignment	Difficult to organize manually	Three defined priority levels
Queue number	May require manual assignment	Automatically generated
Record searching	Manual searching	Search by name or ID
Department management	Separate manual lists	Department filter
Status tracking	Manual updates	Digital status management
Record storage	Paper records	MySQL database
Queue overview	Multiple records or lists	Centralized dashboard

The comparison highlights the main benefits of the proposed system. It provides a more structured approach to patient registration, queue prioritization, record retrieval, and consultation tracking. The system is intended as a demonstration application and requires further evaluation before use in a real hospital environment.

CHAPTER – 7 COMPLEXITY ANALYSIS

Let (n) represent the number of patient records.

Operation	**General Complexity**
Basic record insertion	(O(1)), simplified model
Queue sorting	(O(n\log n))
Search without an applicable index	(O(n))
Filtering without an applicable index	(O(n))
Update using an indexed record key	Typically (O(1)), simplified model

Queue sorting may require (O(n\log n)) time under a comparison-based sorting model. Search and filtering performance depend on the database query, indexes, and execution plan. These are theoretical estimates rather than measured performance results.

CHAPTER – 8 ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

8.1 Advantages

- Faster and more organized patient registration.
- Clear identification of Emergency, Urgent, and Normal priorities.
- Unique queue numbers.
- Easy patient search and department filtering.
- Centralized dashboard and consultation tracking.
- Reduced dependence on manual records.
- Basic authentication and database security measures.
- Light and dark display modes.

8.2 Limitations

- Designed for a single-hospital demonstration.
- No complete staff account approval workflow.
- No rate limiting or dedicated audit log.
- No SMS or WhatsApp notifications.
- No waiting-time estimation or appointment management.
- No advanced analytics.
- Not intended for storing real patient data or production hospital use.

8.3 Future Scope

Future improvements may include doctor-specific login and role-based access, SMS and WhatsApp alerts, waiting-time estimation, appointment scheduling, queue analytics, cloud deployment, staff account approval, audit logging, and rate limiting.

CHAPTER – 9 CONCLUSION

The Smart Patient Queue System provides a centralized web-based approach to hospital queue management. It supports staff authentication, patient registration, automatic queue numbering, priority-based ordering, patient search, department filtering, and consultation-status tracking.

Using Python, Flask, Jinja2, HTML5, CSS3, JavaScript, and MySQL, the project demonstrates how web technologies can help reduce manual work and make patient queues more organized. Emergency patients receive higher queue priority, while registration time determines the order among patients with equal priority.

The current system is a demonstration application. Additional security controls, privacy safeguards, testing, and operational improvements would be required before real hospital deployment.

Overall, the project demonstrates a practical application of web development and database management to improve the organization and visibility of hospital queues.

REFERENCES

1. Python Documentation: https://docs.python.org/
2. Flask Documentation: https://flask.palletsprojects.com/
3. Jinja Documentation: https://jinja.palletsprojects.com/
4. MySQL Reference Manual: https://dev.mysql.com/doc/
5. MDN Web Docs: https://developer.mozilla.org/
6. OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
7. Smart Patient Queue System, Project Seminar Presentation, Department of Information Technology, A. D. Patel Institute of Technology.